
PROJECT-BASED LEARNING

Build a Real AI App

using Node.js + LLM + RAG

Step-by-step notes from a real learning session

Node.js • Express • Gemini AI • RAG • Embeddings • Vector Search

Author

Prince Kumar

YouTube: [Hello World by Prince](#)

LinkedIn: [linkedin.com/in/iamprince](https://www.linkedin.com/in/iamprince)

Instagram: [@helloworldbyprince](#)

Community: [JobEngine — t.me/helloworldbyprince](#)

Build AI Projects Playlist:

<https://www.youtube.com/playlist?list=PLzjZaW71kMwQX6hly20472hPxNaELxo3j>

NodeJS playlist:

<https://www.youtube.com/playlist?list=PLzjZaW71kMwScTRKzoasdyB1sX-a9EbFp>

LLM Bootcamp:

<https://www.youtube.com/playlist?list=PLzjZaW71kMwS2MrPcY22-oZxHjrpi6yEZ>

Github Repo:

<https://github.com/Prince-1501/ai-projects-nodejs>

Project Setup

Run these commands to create the project structure:

```
mkdir chat-with-pdf
cd chat-with-pdf

mkdir server
cd server

npm init -y
```

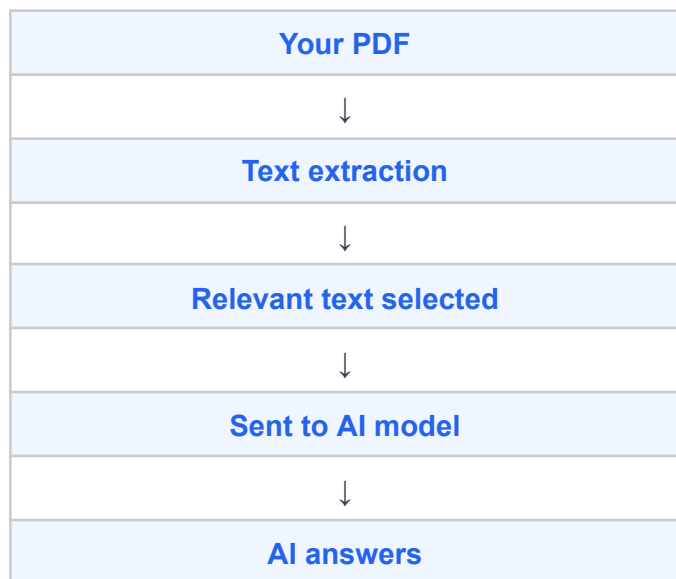
After running this, you will see: package.json

This file stores:

- project info
- dependencies
- scripts

What ChatGPT Does Internally

When you upload PDF in ChatGPT:



Your project will do the SAME concept.

IMPORTANT UNDERSTANDING

AI models like:

- GPT
- Gemini
- Claude

do NOT naturally "open PDFs". They only understand: Text

TEXT

That's it. So every AI PDF app first converts: PDF → Text. Then gives text to AI.

Real World Analogy

Imagine AI is a very smart student... BUT blind to file formats. If you give:

- .pdf
- .docx
- .pptx

AI says: "Bro... convert this into readable text first 🤔"

That's basically what we do.

SUPER IMPORTANT DIFFERENCE

Uploading ≠ Understanding

Just because a user uploads a file... does NOT mean AI understands it. There are multiple hidden steps. That hidden pipeline is what AI engineers build. And THAT is why this project is powerful for learning.

Install Backend Packages

Run:

```
npm install express multer pdf-parse cors
```

Now understand each one simply:

Package	Purpose
express	creates backend server
multer	handles file uploads
pdf-parse	extracts text from PDF
cors	allows frontend/backend communication

What is Express?

Express is a backend framework for Node.js.

Without Express:

```
Node.js = raw engine
```

With Express:

```
Express = easier backend development
```

Similar to:

Thing	Analogy
Node.js	raw kitchen
Express	organized cooking setup

We use Express because building APIs becomes much easier.

Run the install command first.

Now create main backend file

```
touch index.js
```

Then open index.js

Add this:

```
const express = require("express");

const app = express();

app.get("/", (req, res) => {
  res.send("Server is running");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Now run:

```
node index.js
```

Then open:

```
http://localhost:3000
```

You should see:

```
Server is running
```

Now understand this flow:

```
Browser → Request → Express Server → Response
```

Very important terms:

Term	Meaning
Server	program running continuously waiting for requests
Request	browser asking something
Response	server replying
Route	specific backend path like / or /upload

Example:

```
app.get("/")
```

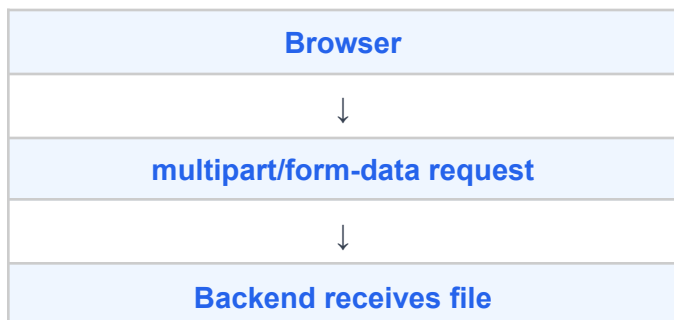
means: if someone visits the homepage, run this code.

File Upload

Now we'll make first real AI-app feature: File Upload

First understand

When user uploads PDF:



Normal JSON requests:

```
{  "name": "prince"}
```

But files cannot be sent like normal JSON. So browser sends:

```
multipart/form-data
```

Think of it like: special package format for files.

Multer

multer helps Express receive uploaded files.

Without multer:

- Express understands JSON
- but NOT file uploads properly

Create Upload Folder

Inside server:

```
mkdir uploads
```

Replace index.js with this

```
const express = require("express");
const multer = require("multer");

const app = express();

const upload = multer({
  dest: "uploads/",
});

app.get("/", (req, res) => {
  res.send("Server running");
});

app.post("/upload", upload.single("pdf"), (req, res) => {
  console.log(req.file);

  res.send("PDF uploaded successfully");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Understand Slowly

This:

```
upload.single("pdf")
```

means: accept ONE uploaded file

And:

```
"pdf"
```

is the field name coming from frontend/Postman.

Very Important

After upload:

```
req.file
```

contains uploaded file information. Like:

- filename
- path
- size
- mimetype

Now run server again:

```
node index.js
```

After that tell me:

- done
- or any error

Then we'll test upload using Postman and understand the request flow deeply.

Test Upload Using Postman

Why Postman?

Normally the frontend sends requests. But the frontend is not built yet. So Postman helps us manually test backend APIs. Think:

```
Postman = fake frontend for testing
```

Steps

1. Open Postman — Create:

```
POST request
```

URL:

```
http://localhost:3000/upload
```

2. Go to Body — Select:

```
form-data
```

Why? Because files use:

```
multipart/form-data
```

NOT raw JSON.

3. Add Key

KEY	TYPE
pdf	File

Important:

- name MUST be pdf
- because backend expects:

```
upload.single("pdf")
```

Both names must match.

4. Choose PDF — Upload any PDF. Then click Send.

Expected Result

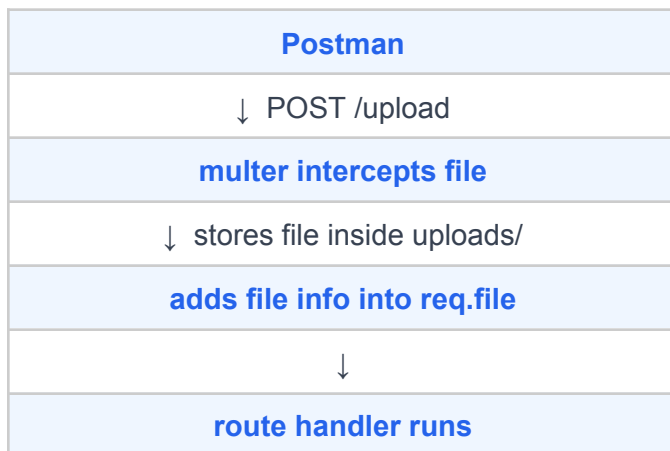
Response:

```
PDF uploaded successfully
```

And terminal should print:

```
req.file object.
```

Understand Flow Carefully



VERY IMPORTANT CONCEPT — Middleware

This:

```
upload.single("pdf")
```

is middleware. Meaning: code that runs BEFORE final route logic.

Flow:



Express middleware is extremely important in backend engineering.

Extract Text from Uploaded PDF

Now we'll do the MOST important part till now: Extract text from uploaded PDF

Currently:

```
PDF uploaded successfully
```

But AI still cannot read it. We need:

```
PDF → readable text
```

Update index.js

Replace /upload route with this:

```
const express = require("express");
const multer = require("multer");
const pdfParse = require("pdf-parse");
const fs = require("fs");

const app = express();

const upload = multer({
  dest: "uploads/",
});

app.get("/", (req, res) => {
  res.send("Server running");
});

app.post("/upload", upload.single("pdf"), async (req, res) => {
  try {
    const dataBuffer = fs.readFileSync(req.file.path);

    const pdfData = await pdfParse(dataBuffer);

    console.log(pdfData.text);

    res.send(pdfData.text);
  } catch (error) {
```

```
    console.log(error);

    res.status(500).send("Error reading PDF");
  }
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Understand Slowly

fs

```
const fs = require("fs");
```

fs means: file system

Node.js module for:

- reading files
- writing files
- deleting files

This line

```
fs.readFileSync(req.file.path)
```

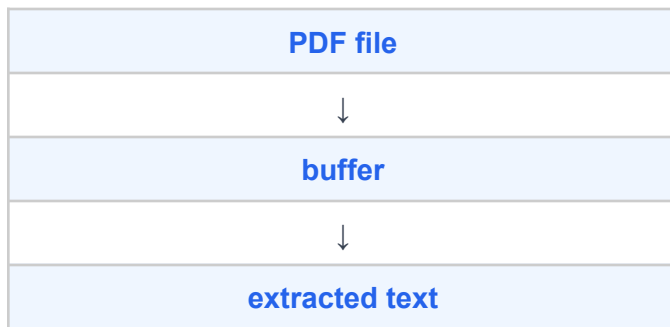
means: read uploaded PDF from uploads folder

Returns: binary file data

Important Difference

Thing	Meaning
File path	location of file
Buffer	raw binary data in memory
Text	human readable content

Flow:



pdfParse()

```
await pdfParse(dataBuffer)
```

This library:

- analyzes PDF structure
- extracts readable text

Returns object like:

```
{ text: "...pdf content..."}
```

Important — Why async/await?

PDF reading takes time. Node.js does not want the server to freeze while waiting. So:

```
await
```

means: wait here until PDF parsing completes

We'll deeply learn async later while building queues and AI calls.

Now:

- restart server
- upload PDF again from Postman

You should now see:

- PDF text in terminal
- PDF text in response

That means: AI-readable text extraction successful

Now restart server:

```
node index.js
```

Then upload the PDF again. You should now get extracted text successfully.

Still getting errors? Use stable version

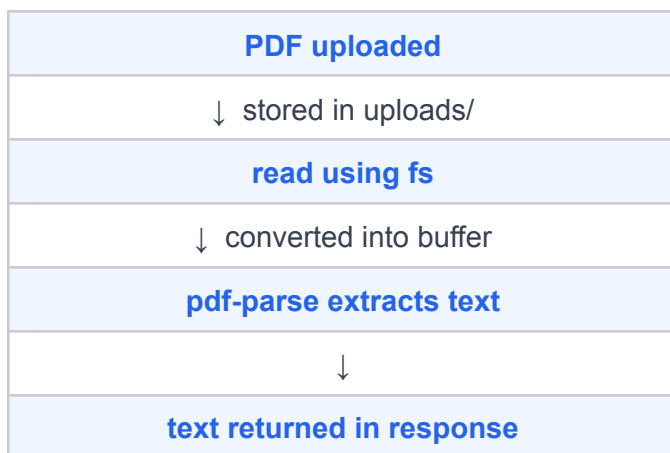
```
npm uninstall pdf-parse  
npm install pdf-parse@1.1.1
```

Then use: `const pdfParse = require("pdf-parse");`
This will work properly with your current CommonJS setup. No other code changes needed.

Then restart: `node index.js`

Now understand what just happened technically

Current Flow



Most Important Learning Till Now

AI is NOT reading PDF directly. Your backend is doing: PDF → Text Then later: Text → AI

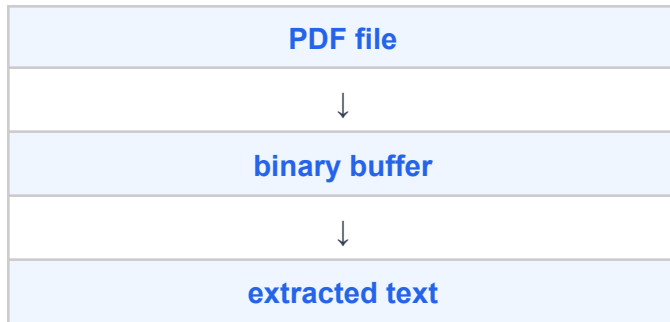
One Important Term — Buffer

You saw:

```
const dataBuffer = fs.readFileSync(req.file.path);
```

A buffer means: raw binary data stored temporarily in memory

Computers internally understand: 0s and 1s — not human-readable PDF directly. So flow becomes:



Term	Meaning
File	stored document on disk
Buffer	raw file data in memory
Text	readable content

Now the next small step. Currently the response returns FULL PDF text. That is bad for AI systems later because:

- PDFs can be huge
- AI models have token limits
- sending entire PDF is expensive

So next we'll learn: Chunking

Meaning: Large text → smaller meaningful pieces

This is one of MOST important RAG concepts.

Chunking

Very simple meaning:

Big text → split into smaller parts

Why do we need chunking?

Imagine PDF has:

- 200 pages
- 50,000 words

You CANNOT send the entire thing to AI every time. Problems:

- expensive
- slow
- token limit issues
- irrelevant information

So instead: Find only relevant small pieces → send those to AI. That's why chunking exists.



Real World Analogy

Instead of giving student: whole textbook you give: only relevant paragraphs

Important Related Terms

Term	Meaning
Chunking	splitting text
Retrieval	finding relevant chunks
Context	chunks sent to AI
Tokens	small pieces AI reads internally

First We'll Do Simple Chunking

Example:

Chunk 1 = first 500 characters
Chunk 2 = next 500
Later we improve it intelligently.

Add This Inside /upload

Replace:

```
console.log(pdfData.text);res.send(pdfData.text);
```

with:

```
const text = pdfData.text;

const chunks = [];

for (let i = 0; i < text.length; i += 500) {
  chunks.push(text.slice(i, i + 500));
}

console.log(chunks);

res.json({
  totalChunks: chunks.length,
  chunks,
});
```

Understand Slowly

text.slice()

```
text.slice(start, end)
```

means: cut part of string

Example:

```
"Hello World".slice(0, 5)
```

Output: Hello

This Loop

```
i += 500
```

means: move 500 characters forward each time

Result

Large PDF becomes:

```
[ "chunk 1...", "chunk 2...", "chunk 3..."]
```

Now upload the PDF again and inspect the response carefully.

IMPORTANT problem with current chunking

Current chunking: 500 characters each. This is called: fixed-size chunking. Simple... but not smart.

Problem Example

Imagine text:

```
HTTP is a protocol used for communication between browser and server...
```

Bad chunking may split like:

```
Chunk 1:"HTTP is a protocol used for"Chunk 2:"communication between browser..."
```

Meaning breaks. AI loses context quality.

Better Chunking Idea

We should split by:

- paragraphs
- sentences
- meaning

NOT random cuts. This is called: semantic chunking. We'll do basic smart chunking now.

Replace chunking code with this

```
const text = pdfData.text;

const chunks = text.split("\n\n");

console.log(chunks);

res.json({
  totalChunks: chunks.length,
  chunks,
});
```

What is \n\n ?

Means: double line break. Usually paragraph separation.

Example:

```
Paragraph 1Paragraph 2
```

Internally becomes:

```
"Paragraph 1\n\nParagraph 2"
```

What is .split() ?

```
text.split(...)
```

means: divide string into array pieces

Example:

```
"apple,banana,mango".split(",")
```

Output:

```
["apple", "banana", "mango"]
```

Why is this chunking better?

Because now chunks preserve meaning better. Instead of: random 500 chars — we get: logical paragraphs. This improves:

- retrieval quality
- AI answers
- context understanding

Upload PDF again and inspect chunks.

What is RAG?

This is exactly where you should stop and understand deeply before moving.

RAG stands for: Retrieval Augmented Generation

Break it slowly:

Word	Meaning
Retrieval	finding relevant information
Augmented	adding extra context
Generation	AI generating answer

Simple Meaning

RAG means: Before AI answers, give it relevant information first.

Without RAG

Normal AI:

Question → AI → Answer

Problem:

- hallucinations
- outdated info
- no access to your PDFs

With RAG

Question → search relevant notes → send notes to AI → AI answers using notes

THIS is RAG.

Real Example

You uploaded networking notes. You ask: "What is HTTP?"

System does:

Step 1 — Retrieval

Find relevant chunk:

```
"HTTP is protocol used between browser and server..."
```

Step 2 — Augmentation

Attach a chunk to the AI prompt. Like:

```
Use this note:"HTTP is protocol..."Now answer:"What is HTTP?"
```

Step 3 — Generation

AI generates the final answer.

Now Embedding Analogy Properly

This is MOST important.

Problem

Computers do NOT understand meaning like humans. For computer:

```
Car and automobile are different words.
```

But humans know: same meaning.

So we need a way to represent MEANING mathematically. That representation is: **embedding**

Meaning Map Analogy

Imagine AI creates invisible maps. Example:

```
Dog Puppy  
Cat Tiger  
Car Bike
```

AI places similar meanings nearby. Like:

```
Dog ---- Puppy
Cat ---- Tiger
Car ---- Bike
```

Not based on spelling. Based on: semantic meaning

Another Example

These:

```
"What is HTTP?"
"Explain browser-server communication protocol"
```

look different... BUT embeddings place them nearby because meaning is similar.

Why This Matters In Our Project

When user asks question: "Explain HTTP"

System searches:

- chunks with similar meaning
- NOT exact same words only

That is why embedding search is powerful.

Why Google Gemini instead of OpenAI?

OpenAI requires a basic fee (~\$5) to use the API. Some students might not have that much money. So we'll use Google Gemini API instead.

Also important: AI Model Provider $\neq$ RAG.

RAG is architecture. You can use Gemini, OpenAI, Claude, or local models with the same RAG pipeline.

Only the model changes. Architecture stays similar.

Gemini has two things we need

Purpose	Model Type
Generate answer	LLM
Convert text to meaning vectors	Embedding model

Before coding embeddings — understand this

Without embeddings:

```
Question:"Explain browser communication"  
Chunk:"HTTP protocol connects client and server"
```

Keyword search may fail because words are different. But embeddings understand:

```
browser communication  $\approx$  client-server protocol
```

That is the REAL magic behind semantic AI search.

Embeddings — What Are They?

Embeddings are: numbers. More specifically: array of numbers.

Example:

```
[0.12, -0.93, 0.44, 0.81, ...]
```

But IMPORTANT: Those numbers represent:

- semantic meaning
- not grammar
- not exact words
- not sentence length

Simple Understanding

AI converts text into: mathematical meaning representation

Example

Sentence:

```
"Dog is a pet"
```

might become:

```
[0.21, -0.55, 0.91, ...]
```

Another sentence:

```
"Puppy is a domestic animal"
```

might become:

```
[0.19, -0.51, 0.88, ...]
```

Notice: numbers become somewhat similar. Because: meanings are similar

Important Clarification

One embedding does NOT represent: all text together always. Usually:

```
1 chunk → 1 embedding vector
```

Example:

Chunk	Embedding
"HTTP is protocol..."	[0.22, 0.91, ...]
"React is frontend..."	[-0.11, 0.45, ...]

Why vectors?

Because mathematics can compare them. AI systems calculate: Which vectors are closest?

```
Closest vectors = closest meanings.
```

VERY IMPORTANT TERM — Vector

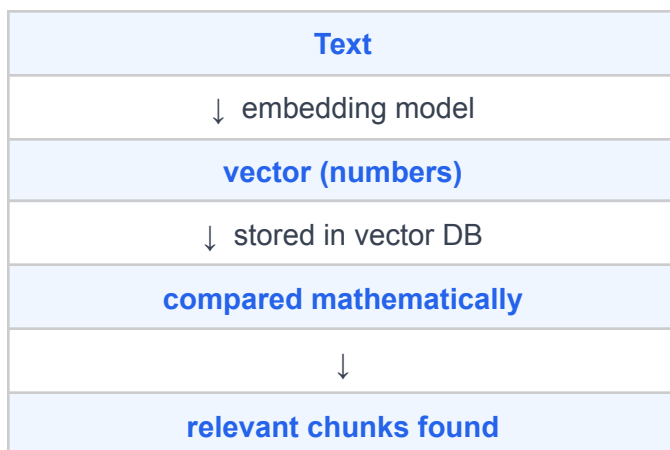
Fancy AI word. But simple meaning: list/array of numbers

Example:

```
[0.12, 0.55, -0.91] That's a vector.
```

Final Understanding

Flow:



This is the core foundation of modern RAG systems.

Forget AI terms for 2 minutes. Let's think like humans first.

Suppose I give you 3 sentences:

1. Dog is a pet
2. Puppy is a domestic animal
3. Car has four wheels

Now as HUMAN you instantly feel: Sentence 1 and 2 are related — even though words are different.

Why? Because your brain understands: meaning — not exact words.

But computer problem

Computer sees:

```
dog ≠ puppy  
pet ≠ domestic animal
```

So computer needs some way to understand: semantic similarity

Embedding Solves This

Embedding models convert text into numbers.

Example:

```
"Dog is a pet" becomes:[0.11, 0.92, -0.44]
```

Another sentence:

```
"Puppy is a domestic animal" becomes:[0.10, 0.89, -0.40]
```

Notice: numbers somewhat similar.

But this:

```
"Car has four wheels" may become:[-0.77, 0.02, 0.91]
```

Very different numbers.

Why Do Similar Numbers Matter?

Because now computer can do: math comparison and detect: Sentence 1 close to Sentence 2

Real Analogy — GPS Coordinates

Sentence	Position
Dog is pet	Delhi
Puppy domestic animal	Noida
Car wheels	Mumbai

Now system says: Delhi close to Noida → meaning: similar meaning

But: Mumbai far away → meaning: different meaning

In YOUR Project

PDF chunk:

```
"HTTP protocol between browser and server"
```

Question:

```
"How browser communicates with server?"
```

Words are different. BUT embeddings place them nearby mathematically. So system understands: same topic

This is the REAL power of embeddings.

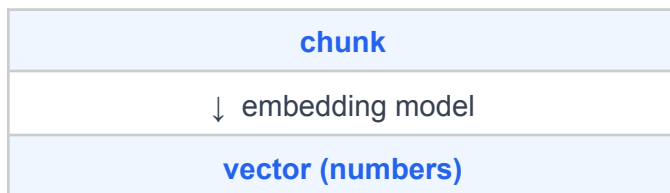
Setting Up Gemini — Generate First Embedding

First Understand Architecture

Current:



Next:



Important

Embedding models do NOT generate answers. It ONLY converts: text → vector.

Install Gemini SDK

Run:

```
npm install @google/genai
```

Next Important Thing

You need: Gemini API Key. From: Google AI Studio — Create free API key.

Then install dotenv

```
npm install dotenv
```

Why dotenv?

Instead of writing secret key directly:

```
✗ bad:const API_KEY = "abcd123";
```

we store in: .env. This keeps secrets separate from code.

Create .env — Inside server/

```
GEMINI_API_KEY=your_api_key_here
```

Do these 3 steps only:

- install Gemini SDK
- install dotenv
- create .env

Then we'll generate the first embedding carefully.

Google SDK Update

Google is moving from @google/generative-ai to @google/genai.
So we'll use the latest SDK.

Run:

```
npm uninstall @google/generative-ai  
npm install @google/genai
```

Then create .env:

```
GEMINI_API_KEY=your_api_key_here
```

SDK	Status
@google/generative-ai	older
@google/genai	latest official SDK

Good engineering habit:

- use maintained SDKs
- avoid deprecated packages for new projects

Now generate first embedding

First Important Reality

Gemini SDK mainly focuses on:

- generation
- chat
- multimodal

So first we'll do: simple text generation. Then embeddings.

Reason:

- easier understanding
- confidence building
- fewer SDK/version issues

Update index.js

Add at top:

```
require("dotenv").config();

const { GoogleGenAI } = require("@google/genai");
```

Then below imports:

```
const ai = new GoogleGenAI({
  apiKey: process.env.GEMINI_API_KEY,
});
```

Now Replace Route Response

Inside /upload — Replace:

```
res.json({
  totalChunks: chunks.length,
  chunks,
});
```

with:

```
const response = await ai.models.generateContent({
  model: "gemini-2.5-flash",
  contents: `
```

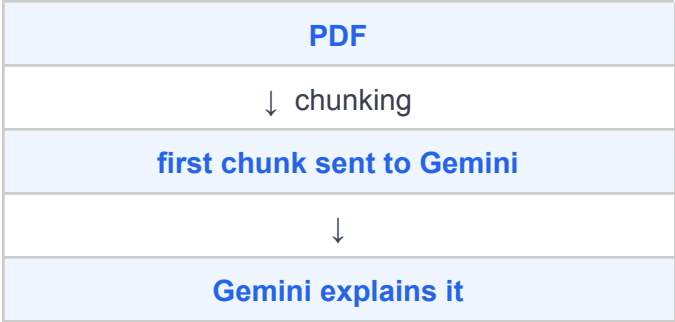
```
    Explain this PDF content in simple words:

    ${chunks[0]}
    `
  },
});

res.send(response.text);
```

What is happening?

Flow:



Thing	Purpose
Embedding model	converts text → vectors
LLM (Gemini)	generates human-like text

Right now we are using: LLM. NOT embeddings yet. Because first you should see: our backend talking to AI

Upload PDF again after restarting server.

Why Embeddings Exist Separately — Deep Dive

YES — Embeddings ALSO require AI models. Because: understanding semantic meaning = AI task

Two Different AI Jobs

- 1. Text Generation Model (LLM)
- Purpose: generate human-like text
- Example: ChatGPT, Gemini, Claude

```
"What is HTTP?"→"HTTP is protocol..."
```

2. Embedding Model

Purpose: convert text into vectors (numbers)

```
"What is HTTP?" → [0.12, -0.44, 0.91, ...]
```

IMPORTANT

Both are AI models. But trained for DIFFERENT tasks.

Person	Job
Teacher (= LLM)	explains answers
Librarian (= Embedding model)	organizes similar books together

How OpenAI Handles It

OpenAI provides chat models and embedding models separately.

Purpose	OpenAI Model
Chat	GPT-4
Embeddings	text-embedding-3-small

Same company. Different models.

Why Embeddings Exist Separately

Because: generating text ≠ measuring semantic similarity. Different optimization goals.

FINAL UNDERSTANDING — In RAG systems

Step 1 — Embedding model:

```
chunk → vector
```

Step 2 — User question:

```
question → vector
```

Step 3 — Similarity search:

```
find closest vectors
```

Step 4 — LLM:

```
generate final answer
```

So BOTH models work together in RAG.

RAG vs Directly Sending Whole PDF

Option 1 — Directly Send Whole PDF

```
User question + entire PDF → Gemini → answer
```

This works for small PDFs. BUT huge problems happen.

Problem 1 — Token Limits

AI models cannot take infinite text. They have limits.

```
Huge PDF = too much text → Model may reject request.
```

Problem 2 — Expensive

```
more tokens → more cost
```

Sending a 200-page PDF every question is wasteful.

Problem 3 — Slow

```
Large context → more processing time → Responses slower.
```

Problem 4 — Noise

Suppose the user asks: "What is HTTP?" But PDF contains: DBMS, OS, networking, Java, Linux. AI sees EVERYTHING. Important signal diluted.

RAG Solves This

```
Question → find relevant chunks only → send only useful context → generate answer
```

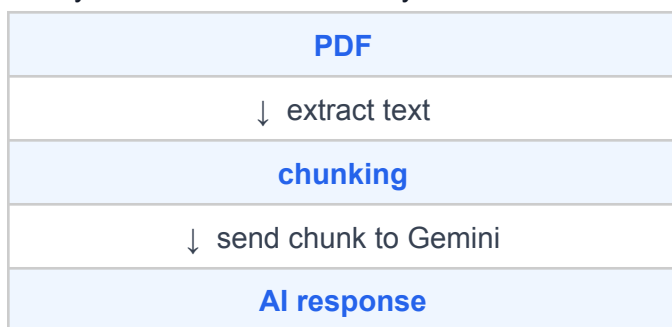
Without RAG	With RAG
Question + entire PDF	Question + relevant chunks only
Token limit problems	Only small relevant pieces sent
Expensive	Cost-efficient
Slow and noisy	Fast and focused

IMPORTANT

RAG is mainly about: retrieval optimization. NOT "making AI smarter". It improves: relevance, scalability, cost, speed. That's why companies use it.

First Real AI Backend Pipeline

Now your backend successfully does:



This is your FIRST real AI backend pipeline.

Now the next important step: Currently AI only sees the first chunk because: `${chunks[0]}`. That is NOT real RAG yet.

Real RAG means: user asks question → system finds relevant chunks dynamically → send those chunks to AI.

So now we need: semantic search — find the most relevant chunk before sending it to AI.

But before embeddings/vector DB... we'll first build: basic keyword search

Why?

- easier understanding
- helps compare keyword vs semantic search
- builds intuition gradually

This is a VERY important learning step.

Note on Model

We are using gemini-2.5-flash-lite (not gemini-2.5-flash).
It is: faster, cheaper, and enough for learning RAG concepts.

Model	Better For
flash-lite	speed + low cost
flash	balanced
pro	deeper reasoning

Keyword Search — Building Intuition

Next we'll do: User asks question → search matching chunks → send relevant chunk to Gemini.
Instead of always: chunks[0]

Use this sample PDF

Use this sample PDF for:

- upload testing
- chunking
- retrieval
- RAG experiments
- keyword search
- embeddings later

Download the sample PDF.

First — Hardcoded Keyword Search

Add this after chunking inside /upload:

```
const question = "What is HTTP?";

const matchedChunk = chunks.find((chunk) =>
  chunk.toLowerCase().includes("http")
);

const response = await ai.models.generateContent({
  model: "gemini-2.5-flash-lite",
  contents: `
    Answer the question using this context:

    ${matchedChunk}

    Question:
    ${question}
  `,
});

res.send(response.text);
```

Now instead of always chunks[0] — we are finding a relevant chunk first. Then sending it to Gemini.

How .find() Works

so .find will go to each and every element of the array even if he got the http in chunk[2]?
YES. .find() checks elements one by one from start. Example:

```
const chunks = [
  "React is frontend",
  "Node.js backend",
  "HTTP protocol works..."
];

chunks.find((chunk) =>
  chunk.toLowerCase().includes("http")
);
```

Flow:

```
chunk[0] → contains "http" ? NO
chunk[1] → contains "http" ? NO
chunk[2] → contains "http" ? YES
STOP
RETURN THIS CHUNK
```

Important

.find() stops at the FIRST match. It does NOT continue after finding a match. So: efficient enough for small arrays

Difference

Method	Returns
.find()	first matching value
.filter()	ALL matching values

Example:

.find()

```
["a", "http", "http2"]
returns:"http"
```

.filter()

```
returns: ["http", "http2"]
```

Right now .find() is okay because: we only want one relevant chunk. Later in real RAG:

- we retrieve multiple chunks
- rank similarity
- send top chunks to AI

Now we improve retrieval — Make it Dynamic

Current problem: .includes("http") is hardcoded. Bad because:

- fixed question
- fixed keyword
- not dynamic

Now we simulate: real user question

Replace:

```
const question = "What is HTTP?";

const matchedChunk = chunks.find((chunk) =>
  chunk.toLowerCase().includes("http")
);
```

with:

```
const question = req.body.question;

const matchedChunk = chunks.find((chunk) =>
  chunk.toLowerCase().includes(question.toLowerCase())
);
```

Test dynamic questions from Postman

Important

Currently you are sending: form-data — because PDF upload exists. So questions should ALSO go in the same form-data. NOT raw JSON.

Problem

req.body.question may not work properly yet with multer form-data confusion. So first add:

```
app.post("/upload", upload.single("pdf"), async (req, res) => {  
  console.log(req.body);  
  ...  
});
```

Then in Postman — Body → form-data — Add:

KEY	TYPE	VALUE
pdf	File	your PDF
question	Text	http

Important Understanding

Because request contains: file + text — browser/Postman sends: multipart/form-data — NOT JSON. But multer can parse both: uploaded files AND normal text fields. So:

- req.file contains PDF.
- req.body.question contains question text.

Test this first:

- upload PDF
- send question field
- check terminal req.body

Then proceed.

Flow Now



This is the first mini-RAG style flow.

IMPORTANT weakness in current retrieval

Current logic: `.includes(question)`

Example:

```
Question:"What is HTTP?"  
Chunk:"HTTP protocol connects browser and server"
```

Problem: full sentence $\neq$ exact chunk text. So retrieval may fail.

Why? Because keyword matching is: exact-text based. NOT meaning based.

This is where **embeddings** become useful. Embeddings allow:

```
"What is HTTP?" $\approx$ "HTTP protocol communication"
```

Meaning similarity. NOT exact wording.

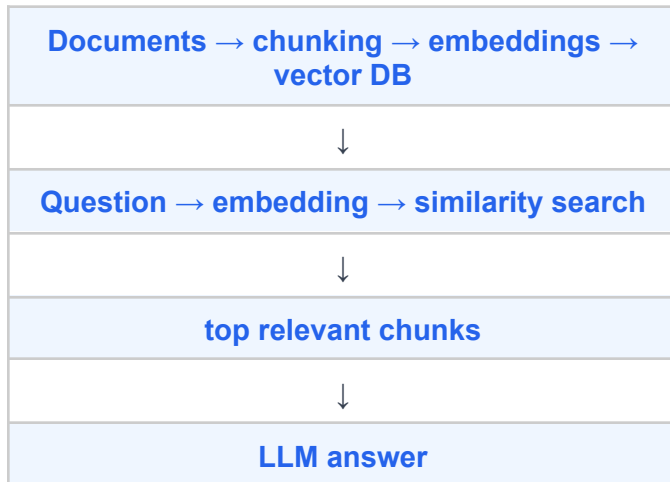
Retrieval — The HEART of RAG

Retrieval means: finding relevant information before AI answers. This is the HEART of RAG.

Current Retrieval Type: keyword retrieval

Later: semantic retrieval using embeddings.

Real Industry Flow



You are gradually building toward this architecture.

Generating Embeddings

Google updated embedding model names. Use this exact function now:

```
async function createEmbedding(text) {
  const response = await ai.models.embedContent({
    model: "gemini-embedding-2",
    contents: text,
  });

  return response.embeddings[0].values;
}
```

Model	Purpose
gemini-2.5-flash-lite	generate answers
gemini-embedding-2	generate embeddings

Two different AI models. Two different jobs.

Now after upload:

```
const embedding = await createEmbedding(chunks[0]);

console.log(embedding.slice(0, 10));
```

Use `.slice(0, 10)` because embedding vectors are HUGE. This prints first 10 numbers only.

Error Fix — Empty Chunks

If you get: `EmbedContentRequest.content` contains an empty `Part`. This means `chunks[0]` is empty. Because `text.split("\n\n")` sometimes creates empty chunks. Fix it:

```
const chunks = text
  .split("\n\n")
  .filter((chunk) => chunk.trim() !== "");
```

Understand

.trim()

Removes: spaces, newlines, empty formatting

Example:

```
"  hello  ".trim()  
becomes:hello
```

.filter()

Keep only matching items. So: `chunk.trim() !== ""` means: keep only non-empty chunks

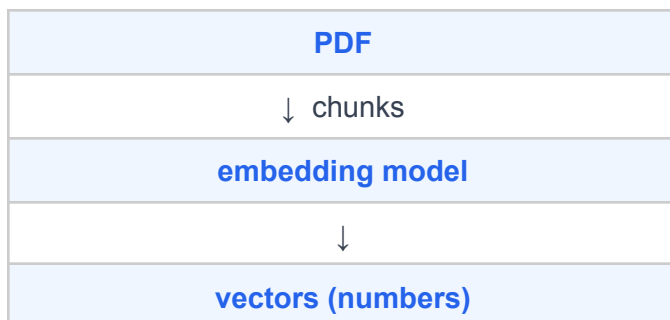
Now retry. Then embedding generation should work properly.

After fix — output printed:

```
[ -0.0021641618, 0.02859029, 0.008048223, -0.009495517, 0.011258917,  
0.0021228741, -0.010731427, 0.010818166,
```

Perfect. Now you successfully created: semantic vectors from text.

Current flow



This is a MAJOR step in AI engineering.

Now understand something VERY important

You printed:

```
[ -0.0021,  0.0285,  0.0080]
```

These numbers individually mean NOTHING to humans. Important thing is: relationship between vectors

Real Magic

Chunk about HTTP:

```
"HTTP protocol..."  
vector:[0.11, -0.22, 0.91]
```

User question:

```
"Explain browser-server communication"  
  
vector:[0.10, -0.20, 0.89]
```

Notice: vectors become similar. Why? Because the meanings are similar.

THIS is semantic search

Instead of: matching words
we compare: meaning vectors

Important Term —
Similarity Search: Which vectors are mathematically closest?
Closest vectors = closest meanings.
This is the heart of vector databases.

Now — Embed Every Chunk

Currently you generated embedding for: chunks[0] ONLY. But real RAG needs: every chunk → its own embedding

Why? Because later: user questions will also become embedding. Then we compare: question vector vs all chunk vectors and find the closest meaning.

So now do this — Replace:

```
const embedding = await createEmbedding(chunks[0]);  
  
console.log(embedding.slice(0, 10));
```

with:

```
const chunkEmbeddings = [];  
  
for (const chunk of chunks) {  
  const embedding = await createEmbedding(chunk);  
  
  chunkEmbeddings.push({  
    text: chunk,  
    embedding,  
  });  
}  
  
console.log(chunkEmbeddings[0]);
```

Understand Slowly

for (const chunk of chunks) — Means: loop through every chunk

chunkEmbeddings — We are storing:

```
{ text: "...chunk text...", embedding: [numbers]}
```

Because later:

- we need original text
- AND vector

Both together are important.

Flow Now

Chunk 1 → vector
Chunk 2 → vector
Chunk 3 → vector

This is how vector databases basically store data internally: text + embedding vector

Run this once and inspect the console carefully.

Cosine Similarity — Semantic Retrieval

Now we'll do the MOST important RAG step: **similarity comparison**

Current: all chunks have embeddings

Next: question → embedding → compare with chunk embeddings → find closest chunk

This is REAL semantic retrieval.

First Understand Problem

We now have:

```
Chunk A → [numbers]
Chunk B → [numbers]

Question → [numbers]
```

How do we know: which chunk's meaning is closest?

Answer: cosine similarity. Fancy name. Simple idea: compare direction/pattern of vectors

Real Analogy

Person	Direction
Delhi → Noida	similar
Delhi → Mumbai	different

Cosine similarity measures: how close two vectors are. Closer meaning: higher similarity score

Add This Function — Above routes

```
function cosineSimilarity(vecA, vecB) {
  let dotProduct = 0;

  for (let i = 0; i < vecA.length; i++) {
    dotProduct += vecA[i] * vecB[i];
  }

  return dotProduct;
}
```

Important

This is a simplified similarity calculation for learning. Real cosine similarity also normalizes vector magnitude. We'll improve later.

Understand Slowly — Dot Product

This:

```
vecA[i] * vecB[i]
```

means: multiply corresponding vector numbers. Then sum everything.

Core Idea

If vectors point similarly: score becomes higher

If meanings unrelated: score lower

Now next we'll: embed question, compare against all chunk embeddings, retrieve best chunk automatically.

Ignore the math for now — Simple Analogy

We need some way to answer: Which chunk meaning is closest to user's question?

We already converted:

- chunks → vectors
- question → vector

Now we compare vectors.

Suppose:

```
Question:"How browser communicates with server?" Question vector:[1, 2, 3]
```

```
Chunk 1:"HTTP protocol..." vector:[1, 2, 4] Very similar numbers.
```

```
Chunk 2:"Cooking recipe..." vector:[90, 11, -7] Very different numbers.
```

So the system says: Chunk 1 probably has a similar meaning. That comparison process is: similarity checking

Now proceed — Below your embedding loop add

```
const questionEmbedding = await createEmbedding(question);
```

Now the question ALSO becomes a vector.

Then below that:

```
let bestChunk = null;
let bestScore = -Infinity;
```

Understand

We are preparing to: compare all chunks and track: which chunk got highest similarity score

```
Highest score = most similar meaning.
```

Now add this

```
for (const item of chunkEmbeddings) {
  let score = cosineSimilarity(
    questionEmbedding,
    item.embedding
  );

  if (score > bestScore) {
    bestScore = score;
    bestChunk = item.text;
  }
}
```

Understand Slowly

Each item looks like:

```
{ text: "...chunk text...", embedding: [numbers]}
```

What Loop Does

Take question vector
↓ compare with chunk vector
calculate score
↓ keep best matching chunk

Example

Chunk	Score
HTTP chunk	0.91
React chunk	0.33
Cooking chunk	0.02

System keeps: HTTP chunk — because highest similarity.

VERY IMPORTANT

This is the REAL core of semantic search. Not exact words. Meaning comparison.

Now after loop:

```
console.log(bestChunk);  
console.log(bestScore);
```

Then run again and inspect:

- best chunk selected
- similarity score

This is your first semantic retrieval system.

Vector Length Question

Q: In cosineSimilarity function is there any chance that length of vecA and vecB is different? Then the loop will fail if vec[A] is of high length — or is it all of the same length for sure?

EXCELLENT question — This is actual engineering thinking

Embedding vectors from SAME model are always the same length. Example:

gemini-embedding-2 may always return: 3072 dimensions

So: question embedding length = chunk embedding length

Why? Because embedding model architecture is fixed. Like: same machine → same output format

Important Rule

You MUST compare vectors generated by: same embedding model

Good:

```
Gemini-embedding-2 vs gemini-embedding-2
```

Bad:

```
OpenAI embedding vs Gemini embedding
```

because: dimensions may differ, meaning spaces differ

But Your Concern Is VALID

In real production code: We should still protect against mismatch.

```
if (vecA.length !== vecB.length) {  
  throw new Error("Vector size mismatch");  
}
```

Good backend engineers think like this. You are already starting to think about:

- edge cases
- assumptions
- system safety

That is very important in AI systems + infra engineering.

Complete RAG Pipeline — Final Step

Now the final step. Instead of: hardcoded keyword chunk — we now use: semantically best chunk

Replace your Gemini call with:

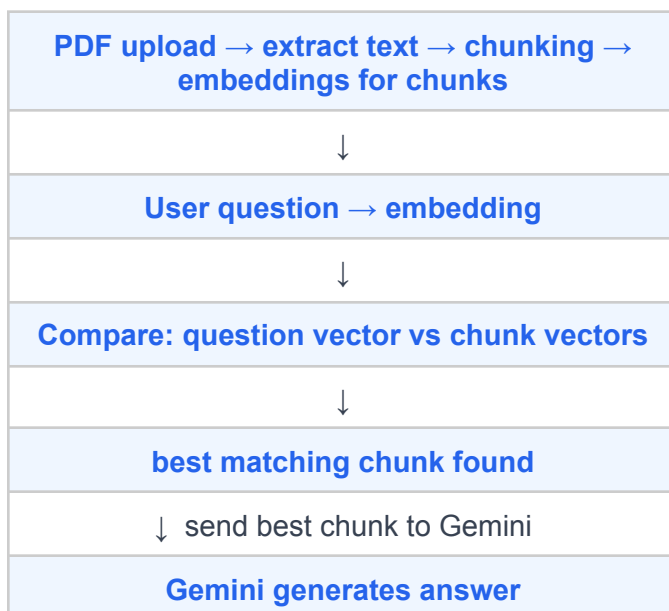
```
const response = await ai.models.generateContent({
  model: "gemini-2.5-flash-lite",
  contents: `
    Answer the question using this context:

    ${bestChunk}

    Question:
    ${question}
  `,
});

res.send(response.text);
```

FULL FLOW NOW



Vector Database — The Next Step

Now we move from: temporary memory → vector database
This is where AI systems become scalable.

Current Problem

Right now:

```
const chunkEmbeddings = []
```

stores vectors in RAM/memory.

Problem:

- When server stops: everything lost
- cannot scale
- slow for huge documents
- difficult searching

What is Vector DB?

Normal DB stores:

Type	Example
text	"Prince"
number	25

Vector DB stores:

```
[0.11, -0.44, 0.91]
```

meaning vectors.

Why Special DB?

Because vector DB can do: similarity search

Example: find top 5 closest vectors — VERY fast. Even among millions of vectors.

Real World Analogy

Normal DB: Find exact item
Vector DB: Find most similar meaning

Popular Vector DBs

DB	Type
Qdrant	open source
Pinecone	cloud
Weaviate	open source
pgvector	PostgreSQL extension

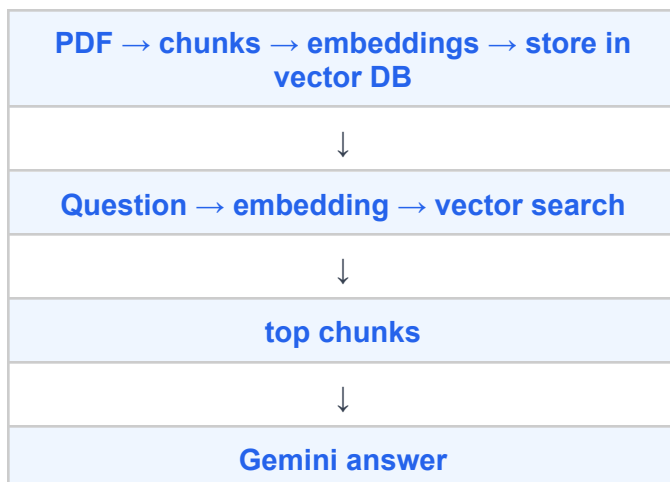
We'll Use — Qdrant

Why?

- beginner friendly
- open source
- real industry usage
- easy API

VERY IMPORTANT UNDERSTANDING

RAG architecture now becomes:



The BIG upgrade is: manual JS loop becomes: database similarity search. Much more scalable.

Connecting Backend to Qdrant

Add Import

At top:

```
const { QdrantClient } = require("@qdrant/js-client-rest");
```

Create Client

Below Gemini client:

```
const qdrant = new QdrantClient({  
  url: process.env.QDRANT_URL,  
  apiKey: process.env.QDRANT_API_KEY,  
});
```

Also update your .env file

Add these two lines to your existing .env:

```
GEMINI_API_KEY=your_api_key_here  
QDRANT_URL=your_qdrant_url  
QDRANT_API_KEY=your_qdrant_api_key
```

Understand

This object:

```
qdrant
```

is now your connection to the vector database. Like:

```
mongoose → MongoDB same idea.
```

Next Important Thing

Vector DB stores data inside: collections

Similar to:

DB	Structure
MongoDB	collection
PostgreSQL	table
Qdrant	collection

Create Collection

Add this temporary route:

```
app.get("/create-collection", async (req, res) => {
  await qdrant.createCollection("pdf-docs", {
    vectors: {
      size: 3072,
      distance: "Cosine",
    },
  });

  res.send("Collection created");
});
```

VERY IMPORTANT

size: 3072 — Means: embedding vector length. Must match embedding model output size.

distance: "Cosine" — Means: compare vectors using cosine similarity. Qdrant will now do similarity searches automatically for us. No manual loop needed later.

Now: restart server. Open browser:

```
http://localhost:3000/create-collection
```

Then tell me your response.

Q: size means length of each vector? How do we know model gives size 3072?

YES. size: 3072 means: length of each embedding vector

Example:

```
[ 0.12, -0.44, 0.91, ...]might contain: 3072 numbersThat is vector dimension size.
```

Why does Qdrant Need This?

Because Qdrant internally prepares storage/indexing structures. It must know: How many numbers each vector will contain?

How Do We Know It Is 3072?

Embedding model documentation usually tells it. OR easiest way:

```
console.log(embedding.length);
```

Run this once. You'll probably see: 3072

Important

All embeddings from SAME model have: same fixed dimension size

Model	Vector Size
gemini-embedding-2	3072
OpenAI small embedding	1536

Different models → different vector dimensions.

Real Analogy

Think: embedding model = machine. Machine always outputs: same number of columns. Like: passport photo always same size. Aadhaar number fixed length. Same idea.

Storing Embeddings in Qdrant

Now we store embeddings inside Qdrant.

Important Understanding

Until now: `chunkEmbeddings[]` was temporary JS memory. Now: vectors go into database

Qdrant Stores Data As

```
{ id, vector, payload}
```

Field	Purpose
id	unique identifier
vector	embedding numbers
payload	extra data like chunk text

Add This Below Embedding Loop

```
const points = chunkEmbeddings.map((item, index) => ({
  id: index + 1,
  vector: item.embedding,
  payload: {
    text: item.text,
  },
}));
```

Understand

We are transforming data into: Qdrant format

Example:

```
{
  id: 1,
  vector: [0.11, 0.44, ...],
  payload: {
    text: "HTTP is protocol..."
  }
}
```

Why Payload?

Because later: vector search finds similar vectors. But the user needs: actual chunk text. Payload stores useful metadata.

Now Add Upload To Qdrant

Below that:

```
await qdrant.upsert("pdf-docs", {
  points,
});
```

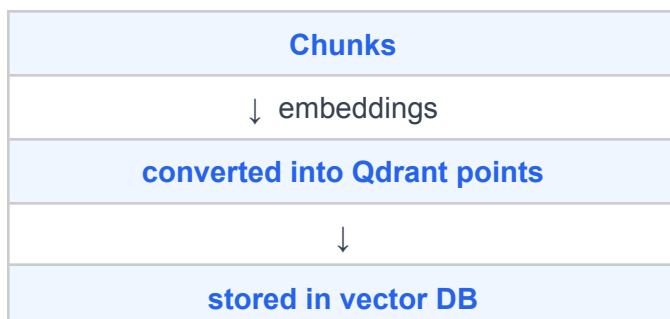
What is Upsert?

Combination of: update + insert

Meaning: store vectors in DB

- If id exists: update
- Else: insert

Flow Now

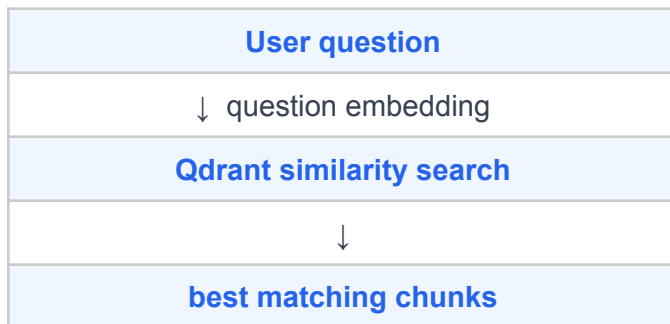


This is a REAL vector storage pipeline.

Semantic Search with Qdrant

Now after vectors are stored... we should NOT manually compare vectors anymore. Qdrant will do a similarity search for us. That is the whole purpose of vector DB.

Next Goal



Add This

Replace your manual similarity loop with:

```
const searchResult = await qdrant.search("pdf-docs", {  
  vector: questionEmbedding,  
  limit: 1,  
});
```

Understand

vector: questionEmbedding — Means: find vectors similar to this question vector

limit: 1 — Means: return top 1 closest chunk

Later: 3, 5, 10 chunks — possible.

What Does Qdrant Return?

Something like:

```
[
  {
    score: 0.91,
    payload: {
      text: "HTTP is protocol..."
    }
  }
]
```

Now Get Best Chunk

Below that:

```
const bestChunk = searchResult[0].payload.text;
```

HUGE THING JUST HAPPENED

Before: YOU manually compared vectors
Now: DATABASE performs semantic search
This is real-world AI architecture.

Now replace Gemini context

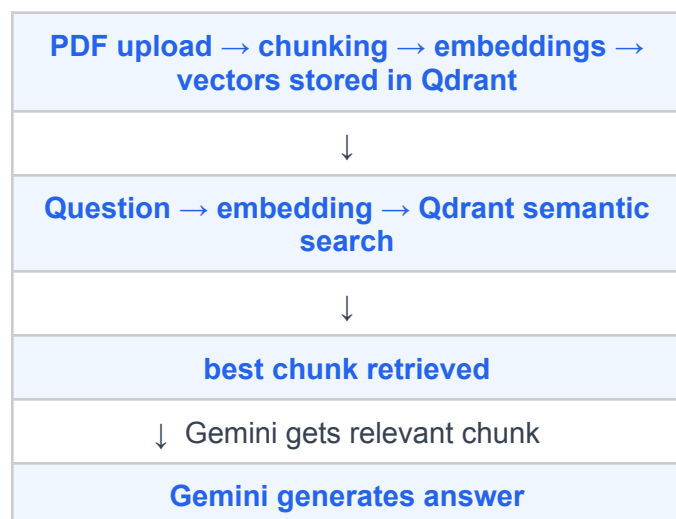
Instead of:

```
${matchedChunk}
```

use:

```
${bestChunk}
```

Final Flow Now



THIS is proper RAG architecture now.

Not beginner fake RAG. Actual semantic retrieval pipeline.

One VERY Important Improvement Note

Currently every upload: re-inserts vectors again. Also IDs: 1, 2, 3 will conflict for multiple PDFs. Production systems use: UUIDs, document IDs, metadata, namespaces. But for learning: current setup is enough.

Next BIG Production Features Possible

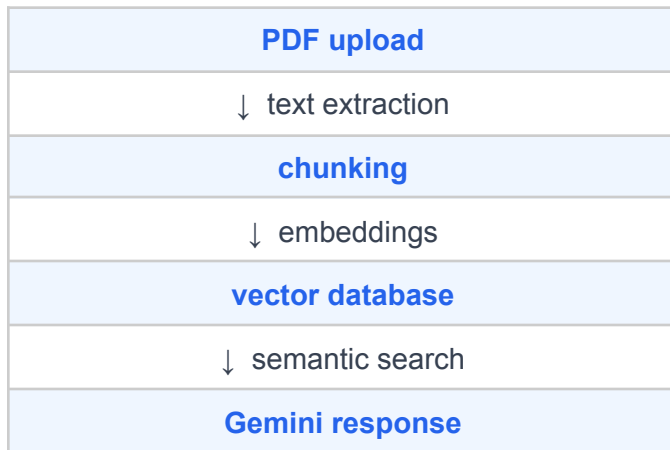
Feature	Why Important
top-k retrieval	multiple chunks
metadata filtering	search specific docs
streaming response	ChatGPT-like typing
chat history	memory
async ingestion	huge PDFs
hybrid search	keyword + semantic
reranking	better retrieval quality

Project Complete — What You Actually Built

Core RAG project? YES — completed.

You built a REAL AI system. Not toy prompt engineering.

What You Actually Built



That is the actual production architecture foundation.

Project Complete. 🎉

PDF → Chunks → Embeddings → Semantic Search → AI Answer

You didn't just follow a tutorial.

You understood the internals. That's the real flex.

— Prince Kumar

[Hello World by Prince](#) • [@helloworldbyprince](#) • [JobEngine](#)